

24A – M4 Trading Documents: Implementation Guide

Everything the M4 developer needs: the trading schema as DDL on top of the kernel document base, seed data including the posting map, package layout, the API slice, every command as a handler specification with pseudocode for the four that carry the module (GRN confirmation, allocation, settlement and its reversal, exposure), the consumers, the snapshot contribution and the till-side GRN bundle contract for M6, screens, tests and tickets. Self-contained.

Field	Value
Document	24A – implementation guide for 24 M4 Trading Documents v0.1
Version	v0.1 – for the M4 developer; sections 7.4 and 8.1 are the contract with the till track
Date	9 September 2026
Prerequisite	Sprint 0; M1, M2, M3 contracts frozen; kernel document base and links (19A §7); M5's availability and lot queries stubbed until doc 25 (Section 11).
Estimated effort	About 32 developer-days for one full-stack developer, or 5 weeks with the driver app screens handed to the till track
Restated content	Marked <i>restated from doc X §Y</i> .

Read this first. M4 is nine document types on one base. Build them in the order they occur in the world (order, delivery, GRN, discrepancy, claim, invoice, payment) and make `ConfirmGrnHandler` the first thing that works end to end with M2 and M5 stubs, because five other things hang off its event. Money is derived from links; if you find yourself storing a balance that isn't recomputed by the same event that changes it, stop.

1. What you are building

Restated from doc 24 §1. The inter-entity flow from order to settlement, recording and linking, never moving money and never blocking on credit.

You build	You do not build
Schema trading: extension tables for ORD, DN (drops), GRN, discrepancy, claim, INV, CN/DN2, PRC (cheque), exposure, posting map, transfer request; the PARTY masking view	The document base, numbering, links, attachments (kernel); stock lots and movements (M5); batch identity (M2); trade prices (M3)
About 40 commands, 12 queries, 12 event families; 5 consumers; the snapshot contribution of expected drop lines; the arbitration queue	The till GRN screen itself (M6 renders it to your specification, Section 8.1); the driver app shell
Eleven web screens and the driver app's drop list and POD screens	Statement rendering engine (kernel A4 renderer; you supply the template)

2. What you inherit

From	You use	How
19A §7 document base	<code>DocumentService.issue(draft, ctx)</code> (validator chain, numbering, hash, state row, audit, events), <code>DocumentLinks.link(from, to, type, amount)</code> , <code>document_state_history</code> , <code>Attachments</code>	Every document type registers a validator and a post-issue hook; every correction is a link
19A §8 sync	Document bundle ingestion calls your <code>GrnPostIngestHook</code> ; <code>SnapshotContributor</code> ; <code>SnapshotChangeLog.record(..., urgent)</code>	Sections 7.3, 7.4
19A §9 attachments	<code>PresignService</code> , attachment status COMPLETE check	Claims and discrepancy evidence
19A §10–12	Notifications (by event rules M9 seeds), <code>ConfigRegistry</code> , <code>@ScheduledJob</code> , <code>@EventConsumer</code>	Windows, escalation clock, statements
M1 api/query	<code>LookupRelationship</code> (terms: window, lock hours, allocation rule, credit limit), <code>listLocations</code> , scope classes, arbiter role	Guards; direct-drop validation
M2 api/query	<code>RegisterBatch</code> (internal command), <code>GetSku</code> flags, conversions, supplier table	GRN confirmation
M3 query	<code>ResolveTradePrice(relationship, sku, uom, qty, date)</code> , adjustment types, <code>TaxQueries.rateInForce</code> via M2	Invoice lines, order adjustments
M5 query (stub until doc 25)	<code>Availability(seller locations, skus)</code> , <code>LotsConsumed(grnId)</code> , <code>PickBatches(dn lines)</code>	Allocation, reversal guard, DN batches
Shared engine	Money, Quantity, Tax for invoice arithmetic	Never re-implement rounding
17A §7 web shell	<code>DocumentHeader</code> , <code>DocumentLines</code> , <code>StateChip</code> , <code>Timeline</code> , <code>ApprovalBar</code> , <code>ReasonCapture</code> , <code>AttachmentCapture</code> , <code>FreshnessStamp</code> , <code>StepUpDialog</code>	Section 8

3. Schema

Restated from doc 24 §3 (CR-24-1). All extension tables key on document_id of the kernel base and are partitioned alike by received_at (carried as a column for the partition key). Location db/migration/m4trading/.

```
-- V0001_trading.sql (partition clauses and RLS lines abbreviated: every table follows 17A 6.3 plus the PARTY
policy)
CREATE TABLE trading.doc_order (document_id uuid NOT NULL, received_at timestamptz NOT NULL, relationship_id
    uuid NOT NULL,
    buyer_entity_id uuid NOT NULL, seller_entity_id uuid NOT NULL, requested_eta date, committed_eta date,
    lock_at timestamptz,
    lock_override_by uuid, allocation_run_id uuid, version smallint NOT NULL DEFAULT 1, owner_entity_id uuid NOT
    NULL,
    PRIMARY KEY (received_at, document_id)) PARTITION BY RANGE (received_at);
CREATE TABLE trading.doc_order_line (line_id uuid NOT NULL, received_at timestamptz NOT NULL, document_id uuid
    NOT NULL,
    requested_qty numeric(14,3) NOT NULL CHECK (requested_qty > 0), allocated_qty numeric(14,3) NOT NULL DEFAULT
    0,
    fulfilled_qty numeric(14,3) NOT NULL DEFAULT 0, cancelled_qty numeric(14,3) NOT NULL DEFAULT 0,
    tier_price numeric(14,4), adjustment_type varchar(24), adjustment_value numeric(14,4), owner_entity_id uuid
    NOT NULL,
    CHECK (allocated_qty + cancelled_qty <= requested_qty), CHECK (fulfilled_qty <= allocated_qty),
    PRIMARY KEY (received_at, line_id)) PARTITION BY RANGE (received_at);
CREATE TABLE trading.allocation_run (run_id uuid PRIMARY KEY, seller_entity_id uuid NOT NULL, ran_at
    timestamptz NOT NULL,
    rule text NOT NULL, availability_snapshot jsonb NOT NULL, order_ids uuid[] NOT NULL, overrides jsonb NOT
    NULL DEFAULT '[]',
    ran_by uuid NOT NULL, owner_entity_id uuid NOT NULL);

CREATE TABLE trading.doc_delivery (document_id uuid NOT NULL, received_at timestamptz NOT NULL,
    seller_entity_id uuid NOT NULL,
    vehicle_ref text, driver_user_id uuid, driver_name text, dispatched_at timestamptz, route_ref text,
    owner_entity_id uuid NOT NULL,
    PRIMARY KEY (received_at, document_id)) PARTITION BY RANGE (received_at);
CREATE TABLE trading.doc_delivery_drop (drop_id uuid NOT NULL, received_at timestamptz NOT NULL, document_id
    uuid NOT NULL, seq smallint NOT NULL,
    ship_to_location_id uuid NOT NULL, bill_to_entity_id uuid NOT NULL, order_ids uuid[] NOT NULL,
    status text NOT NULL DEFAULT 'PLANNED' CHECK (status IN ('PLANNED','DELIVERED','UNDELIVERED','RECEIVED')),
    arrived_at timestamptz, pod_kind text CHECK (pod_kind IN ('PAPER','APP')), pod_signature_key text,
    pod_photo_key text,
    delivered_by uuid, undelivered_reason text, grn_document_id uuid, owner_entity_id uuid NOT NULL,
    PRIMARY KEY (received_at, drop_id), UNIQUE (received_at, document_id, seq)) PARTITION BY RANGE
    (received_at);
-- DN lines: kernel document_line with drop_id and batch_id in the extension below
CREATE TABLE trading.doc_delivery_line (line_id uuid NOT NULL, received_at timestamptz NOT NULL, drop_id uuid
    NOT NULL,
    order_line_id uuid, dispatched_qty numeric(14,3) NOT NULL, owner_entity_id uuid NOT NULL, PRIMARY KEY
    (received_at, line_id)) PARTITION BY RANGE (received_at);

CREATE TABLE trading.doc_grn (document_id uuid NOT NULL, received_at timestamptz NOT NULL, receiver_entity_id
    uuid NOT NULL,
    receiver_location_id uuid NOT NULL, seller_entity_id uuid, drop_id uuid, delivery_document_id uuid,
    purchase_order_ref text,
    supplier_id uuid, received_on date NOT NULL, counted_by uuid, confirmed_by uuid, confirmed_at timestamptz,
    reversal_of uuid,
    owner_entity_id uuid NOT NULL, PRIMARY KEY (received_at, document_id)) PARTITION BY RANGE (received_at);
CREATE TABLE trading.doc_grn_line (line_id uuid NOT NULL, received_at timestamptz NOT NULL, expected_qty
    numeric(14,3),
    received_qty numeric(14,3) NOT NULL CHECK (received_qty >= 0), damaged_qty numeric(14,3) NOT NULL DEFAULT 0
    CHECK (damaged_qty <= received_qty),
    batch_no varchar(40), manufacture_date date, expiry_date date, printed_mrp numeric(14,2), unit_cost
    numeric(14,4),
    owner_entity_id uuid NOT NULL, PRIMARY KEY (received_at, line_id)) PARTITION BY RANGE (received_at);

CREATE TABLE trading.doc_discrepancy (document_id uuid NOT NULL, received_at timestamptz NOT NULL,
    grn_document_id uuid NOT NULL,
    delivery_document_id uuid, kind text NOT NULL CHECK (kind IN ('SHORT','OVER','DAMAGED','MIXED')),
    window_ends_at timestamptz NOT NULL,
    proposal jsonb, proposed_by uuid, resolution text, resolved_by uuid, resolved_at timestamptz, escalated_at
    timestamptz,
    arbiter_user_id uuid, arbitration jsonb, owner_entity_id uuid NOT NULL, PRIMARY KEY (received_at,
    document_id)) PARTITION BY RANGE (received_at);
CREATE TABLE trading.doc_claim (document_id uuid NOT NULL, received_at timestamptz NOT NULL, grn_document_id
    uuid NOT NULL,
    discrepancy_document_id uuid, kind text NOT NULL CHECK (kind IN
    ('DAMAGED','EXPIRED_ON_ARRIVAL','WRONG_GOODS','QUALITY'));
```

```

    inspected_at timestamptz, inspector uuid, findings text, decision text, decided_at timestamptz, decided_by
    uuid,
    return_required boolean NOT NULL DEFAULT false, escalated_at timestamptz, arbiter_user_id uuid,
    owner_entity_id uuid NOT NULL,
    PRIMARY KEY (received_at, document_id)) PARTITION BY RANGE (received_at);

CREATE TABLE trading.doc_invoice (document_id uuid NOT NULL, received_at timestamptz NOT NULL, relationship_id
    uuid NOT NULL,
    seller_entity_id uuid NOT NULL, buyer_entity_id uuid NOT NULL, grn_document_ids uuid[] NOT NULL,
    delivery_document_id uuid,
    seller_vat_no varchar(20) NOT NULL, buyer_vat_no varchar(20) NOT NULL, tax_point_date date NOT NULL,
    due_date date NOT NULL,
    settled_amount numeric(14,2) NOT NULL DEFAULT 0, credited_amount numeric(14,2) NOT NULL DEFAULT 0,
    e_invoice_status text NOT NULL DEFAULT 'NOT_SUBMITTED', dispute_reason text, owner_entity_id uuid NOT NULL,
    PRIMARY KEY (received_at, document_id)) PARTITION BY RANGE (received_at);
-- settled_amount and credited_amount are caches recomputed from document_link by the same handlers that write
    the links (doc 24 9.4)

CREATE TABLE trading.doc_payment_receipt (document_id uuid NOT NULL, received_at timestamptz NOT NULL,
    relationship_id uuid,
    payer_kind text NOT NULL CHECK (payer_kind IN ('ENTITY','CUSTOMER')), payer_entity_id uuid,
    payer_customer_account_id uuid,
    method text NOT NULL CHECK (method IN ('CASH','CHEQUE','TRANSFER','DEPOSIT')), reference text, received_on
    date NOT NULL,
    amount numeric(14,2) NOT NULL CHECK (amount > 0), unapplied_amount numeric(14,2) NOT NULL, owner_entity_id
    uuid NOT NULL,
    PRIMARY KEY (received_at, document_id)) PARTITION BY RANGE (received_at);
CREATE TABLE trading.ch cheque (receipt_document_id uuid PRIMARY KEY, bank text NOT NULL, cheque_no varchar(20)
    NOT NULL, dated date NOT NULL,
    status text NOT NULL DEFAULT 'RECEIVED' CHECK (status IN ('RECEIVED','CLEARED','BOUNCED')), outcome_at
    timestamptz, owner_entity_id uuid NOT NULL);

CREATE TABLE trading.relationship_exposure (relationship_id uuid PRIMARY KEY, seller_entity_id uuid NOT NULL,
    amount numeric(14,2) NOT NULL,
    open_invoices numeric(14,2) NOT NULL, accepted_not_invoiced numeric(14,2) NOT NULL, unapplied_credits
    numeric(14,2) NOT NULL,
    unapplied_receipts numeric(14,2) NOT NULL, as_of timestamptz NOT NULL, freshness timestamptz NOT NULL,
    warned_threshold smallint,
    owner_entity_id uuid NOT NULL);

CREATE TABLE trading.posting_map (doc_type_code varchar(8) NOT NULL, line_kind text NOT NULL, side text NOT
    NULL CHECK (side IN ('SELLER','BUYER')),
    debit_role text NOT NULL, credit_role text NOT NULL, amount_source text NOT NULL, PRIMARY KEY
    (doc_type_code, line_kind, side));

CREATE TABLE trading.transfer_request (request_id uuid PRIMARY KEY, mpcs_entity_id uuid NOT NULL,
    from_location_id uuid NOT NULL,
    to_location_id uuid NOT NULL, lines jsonb NOT NULL, status text NOT NULL DEFAULT 'REQUESTED' CHECK (status
    IN ('REQUESTED','APPROVED','REJECTED')),
    decided_by uuid, decided_at timestamptz, reason text, xfr_document_id uuid, owner_entity_id uuid NOT NULL);

-- PARTY masking view (19A 1): what the counterparty sees of an invoice
CREATE VIEW trading.v_invoice_party AS
    SELECT d.document_id, d.doc_number_display, d.owner_entity_id AS seller, d.counterparty_entity_id AS buyer,
        d.business_date,
        d.net_amount, d.tax_amount, d.gross_amount, i.due_date, i.settled_amount, i.credited_amount,
        i.e_invoice_status
    FROM kernel.document d JOIN trading.doc_invoice i USING (document_id)
    WHERE d.doc_type_code = 'INV' AND (d.owner_entity_id = kernel.scope_entity() OR d.counterparty_entity_id =
        kernel.scope_entity()
        OR kernel.scope_class() = 'FEDERATION_VIEW');
-- unit_cost and internal notes are never in the view; lines are exposed through v_invoice_line_party (sku,
    qty, unit_price, tax, total)

```

3.1 Seed data

```

# seed/m4/permissions.yaml (appended): ord.order.draft/submit/accept/lock_override (ENTITY);
    del.note.draft/issue/dispatch (ENTITY);
# del.pod.record (LOCATION, offline); shop.grn.confirm (LOCATION, offline); whs.grn.confirm (ENTITY);
    shop.grn.reverse (ENTITY, mfa);
# del.discrepancy.resolve/escalate (ENTITY); del.claim.raise/inspect/decide/escalate (ENTITY); fed.arbitrate
    (F, mfa);
# bil.invoice.issue/dispute (ENTITY); bil.creditnote.issue (ENTITY, mfa); bil.debitnote.issue (ENTITY);
    bil.payment.record (ENTITY);
# bil.payment.reverse (ENTITY, mfa); bil.statement.generate (ENTITY); shop.transfer.request (LOCATION);
    mpcs.transfer.approve (ENTITY)
# seed/m4/posting-map.yaml (account roles, mapped to a chart of accounts when the accounting system is chosen
    - J-02)
- {doc: INV, line: GOODS,      side: SELLER, debit: RECEIVABLE,      credit: REVENUE,      amount: net}

```

```

- {doc: INV, line: GOODS, side: SELLER, debit: RECEIVABLE, credit: VAT_OUTPUT, amount: tax}
- {doc: INV, line: ADJUSTMENT, side: SELLER, debit: RECEIVABLE, credit: REVENUE, amount: net}
- {doc: GRN, line: GOODS, side: BUYER, debit: INVENTORY, credit: GRN_ACCRUAL, amount: cost}
- {doc: INV, line: GOODS, side: BUYER, debit: GRN_ACCRUAL, credit: PAYABLE, amount: net}
- {doc: INV, line: GOODS, side: BUYER, debit: VAT_INPUT, credit: PAYABLE, amount: tax}
- {doc: CN, line: GOODS, side: SELLER, debit: REVENUE, credit: RECEIVABLE, amount: net}
  # and VAT_OUTPUT reversal
- {doc: DN2, line: CHARGE, side: SELLER, debit: RECEIVABLE, credit: OTHER_INCOME, amount: net}
- {doc: PRC, line: RECEIPT, side: SELLER, debit: BANK_OR_CASH, credit: RECEIVABLE, amount:
  applied}
- {doc: PRC, line: REVERSAL, side: SELLER, debit: RECEIVABLE, credit: BANK_OR_CASH, amount:
  applied}
# seed/m4/config.yaml: trading.availability_mode INDICATIVE | backorder_review_days 30 | grn_reversal_hours 24
| claim_window_days 14 |
# escalation_grace_days 3 | invoice_consolidation PER_GRN | exposure_warn_thresholds [80,100] |
statement_frequency MONTHLY | tier_basis ORDERED_QTY
# seed/m4/notification-rules.yaml (handed to M9): order.accepted -> buyer stores; delivery_note.dispatched ->
buyer stores;
# discrepancy.raised -> seller sales; claim.* -> both; exposure.warning -> seller accounts;
statement.generated -> buyer accounts

```

4. Package layout

```
lk.coopfed.knoweb.m4trading/ @ApplicationModule(allowedDependencies =
    {"kernel", "m1party::api", "m1party::query", "m2catalogue::api",
    "m2catalogue::query", "m3pricing::query", "m5inventory::query"})
api/      commands (~40), events (12 families), value types (Drop, Allocation, Settlement, Resolution)
query/    OrderQueries, DeliveryQueries, GrnQueries, DisputeQueries, BillingQueries, ExposureQueries,
    PostingQueries
internal/
  order/   Order aggregate over the base;
    Create/Submit/Amend/Cancel/Accept/Reject/Allocate/LockOverride/Adjust handlers; AllocationEngine;
    LockClockJob
  delivery/ Delivery, Drop; Create/Issue/Dispatch/Pod/Undelivered handlers; DeliveryValidator
  grn/      Grn; Capture/Confirm/Reverse handlers; GrnValidator; GrnPostIngestHook (bundle path);
    DiscrepancyDetector
  dispute/  Discrepancy, Claim; handlers; EscalationClockJob; ArbitrationService
  billing/  Invoice, CreditNote, DebitNote, PaymentReceipt, Cheque; handlers; InvoiceBuilder (M3 prices,
    tax); SettlementService
  exposure/ ExposureRecalculator (consumer), ExposureWarnings
  statement/ StatementJob, StatementBuilder (template for A4Renderer)
  posting/  PostingMapper (doc -> journal lines by role), PostingsReadyPublisher
  transfer/ TransferRequest handlers
  snapshot/ DropSnapshotContributor (expected lines for shops), OrderSuggestionsContributor (P2
    placeholder)
  queries/  *QueriesImpl
web/       OrderController, DeliveryController, DriverController (driver API), GrnController,
    DisputeController,
    BillingController, StatementController, TransferController, ArbitrationController
```

5. API slice: openapi/m4trading.yaml (excerpt)

```
/v1/trading/orders      post createOrder | get listOrders(role=buyer|seller, status, cursor)
/v1/trading/orders/{id}/submit|amend|cancel|accept|reject|allocate|lock-override|adjustments post ...
/v1/trading/orders/availability get sellerAvailability(sellerId, skuIds[])
/v1/trading/delivery-notes post createDeliveryNote {orderIds[], drops[{shipToLocationId,
    billToEntityId, lines[{orderLineId, batchId, qty}]}]}
/v1/trading/delivery-notes/{id}/issue|dispatch post
/v1/trading/drops/{dropId}/pod post recordProofOfDelivery {kind, signatureKey?, photoKey?, at, lat?,
    lng?} (driver API; idempotent)
/v1/trading/drops/{dropId}/undelivered post
/v1/trading/grns post captureGrn {dropId? | supplierId?, locationId, lines[{skuId, uom,
    expectedQty?, receivedQty, damagedQty, batchNo?, expiryDate?, printedMrp?, unitCost?}]}
/v1/trading/grns/{id}/confirm|reverse post
/v1/trading/discrepancies/{id}/acknowledge|propose|accept|escalate|arbitrate post
/v1/trading/claims post raiseClaim {grnId, kind, lines[], attachmentIds[]}
    /{id}/inspect|approve|reject|escalate|arbitrate post
/v1/trading/invoices post issueInvoice {grnIds[] | deliveryNoteId, consolidation} get
    listInvoices(relationshipId, status)
/v1/trading/invoices/{id}/dispute|resolve-dispute post
/v1/trading/credit-notes post issueCreditNote {invoiceId?, lines[] | amount, reason}
    /{id}/apply post {invoiceId, amount}
/v1/trading/debit-notes post
/v1/trading/payment-receipts post recordPaymentReceipt {relationshipId, method, amount, reference,
    receivedOn, cheque?, settlements[{invoiceId, amount}]}
/v1/trading/payment-receipts/{id}/apply|dispute|reverse|cheque-outcome post
/v1/trading/relationships/{id}/exposure get /statement?period= get | post generate
/v1/trading/transfer-requests post | /{id}/approve|reject post
# example - captureGrn at a shop (the same shape M6 puts in the offline bundle, Section 7.4)
{ "dropId": "...", "locationId": "...", "receivedOn": "2026-10-14",
  "lines": [ { "skuId": "...", "uom": "CASE", "expectedQty": 10, "receivedQty": 10, "damagedQty": 0, "batchNo":
    "B2411A", "expiryDate": "2027-03-31", "printedMrp": 1080.00 },
    { "skuId": "...", "uom": "EA", "expectedQty": 48, "receivedQty": 46, "damagedQty": 2, "batchNo":
    "R77", "expiryDate": null, "printedMrp": null } ] }
```

6. Handler specifications

Command	Guards (in order)	Mutation and effects	Events
CreateOrder	buyer scope; relationship ACTIVE (M1); lines: sku active, uom valid, qty > 0	kernel draft document + doc_order	–
SubmitOrder	DRAFT; ≥1 line; DocumentService.issue (buyer ENTITY series)	SUBMITTED	order.submitted.v1
AcceptOrder	seller scope; SUBMITTED; allocations ≤ requested and ≤ availability snapshot; eta ≥ today; overrides carry reasons	ACCEPTED; allocation_run; lock_at; tier_price per line (M3, ordered qty); exposure event	order.accepted.v1; order.allocated.v1
RejectOrder / CancelOrder	status; reason; cancel only the undispached remainder	status; cancelled_qty	order.rejected/cancelled.v1
AmendOrder	buyer; before lock_at or override; not FULFILLED	version+1; lines superseded (SUPERSEDES link on line set); re-accept required	order.amended.v1
OverrideLockWindow	seller; reason	lock_at; lock_override_by	order.lock_override.v1
RecordOrderAdjustment	Federation→distributor order; type ACTIVE (M3); within bounds; caller holds the type's permission	ADJUSTMENT line	order.adjusted.v1
CreateDeliveryNote	seller; orders ACCEPTED/LOCKED; lines ≤ allocated – fulfilled; drops' ship-to belongs to bill-to (M1); batches from PickBatches (M5) or keyed	draft DN + drops + lines	–
IssueDeliveryNote	valid; issuance (seller series)	ISSUED; order lines fulfilled_qty += dispatched; order status PARTIALLY/FULFILLED; change-log to each drop's shop (expected lines; urgent if ETA within 1 day)	delivery_note.issued.v1 (M5 picks)
Dispatch	ISSUED; vehicle/driver	IN_TRANSIT	delivery_note.dispatched.v1
RecordProofOfDelivery	drop PLANNED; driver scope or paper by dispatcher; idempotent by drop	DELIVERED; keys	drop.delivered.v1
MarkDropUndelivered	reason	UNDELIVERED; DN closes when all drops terminal	drop.undelivered.v1 (M5 return to stock)
CaptureGrn	receiver scope; drop belongs to a location in scope (or local supply with supplier); expected lines from the drop	draft GRN + lines	grn.captured.v1
ConfirmGrn	Section 6.1	the pivot	grn.confirmed.v1; discrepancy.raised.v1?
ReverseGrn	supervisor; within grn_reversal_hours; !inventory.lotsConsumed(grn); no invoice references; reason; MFA	negated GRN issued; REVERSES link; original REVERSED	grn.reversed.v1 (M5 reverts lots)
Acknowledge / Propose / Accept resolution	party scope; status; proposal by one party, acceptance by the other; agreed lines ≤ variance	UNDER_REVIEW / proposal stored / RESOLVED with consequences: ACCEPT_SHORT → IssueCreditNote (seller, in the same transaction, MFA waived because the proposal was accepted by the buyer: DR-1 of this guide); ACCEPT_OVER → extra invoice line or return request	discrepancy.*.v1
EscalateDiscrepancy / ArbitrateDiscrepancy	either party with reason; arbiter F with MFA; binding decision lines	ESCALATED / RESOLVED with arbitration	discrepancy.escalated/arbitrated.v1
RaiseClaim	buyer; GRN CONFIRMED within claim window; lines ≤ received; attachments referenced	issued claim (buyer series) DISPUTES-linked to GRN	claim.raised.v1
InspectClaim / ApproveClaim / RejectClaim	seller; all attachments COMPLETE; approve issues credit note (CREDITS) and sets return_required	status; credit note	claim.*.v1 (M5 return on approved with return_required)
IssueInvoice	seller; GRNs CONFIRMED/CLOSED, not invoiced, same buyer; VAT numbers present (M1); InvoiceBuilder: lines from received_qty at M3 trade price (tier by ordered qty) + adjustments; tax per line; issuance (seller series); snapshot hash	INV ISSUED; doc_invoice; postings_ready	invoice.issued.v1; journal.postings_ready.v1; exposure

Command	Guards (in order)	Mutation and effects	Events
DisputeInvoice / ResolveInvoiceDispute	buyer with reason / seller+buyer or arbiter	DISPUTED / back	invoice.disputed/resolved.v1
IssueCreditNote	seller; MFA (except discrepancy-accepted path); invoice open balance ≥ amount when linked; reason	CN issued; CREDITS link; invoice credited_amount cache; postings	credit_note.issued.v1
ApplyCreditNote / IssueDebitNote	open balance; reason	links; caches	credit_note.applied.v1 / debit_note.issued.v1
RecordPaymentReceipt	Section 6.3	–	payment_receipt.recorded.v1
ApplyReceipt / DisputePayment / ReversePaymentReceipt / RecordChequeOutcome	Section 6.3	–	payment_receipt.*.v1 ; cheque.*.v1
GenerateStatement	party; period	StatementBuilder → A4Renderer → object key	statement.generated.v1
RequestLateralTransfer / Approve / Reject	shops of one MPCS; MPCS scope; availability (M5)	transfer_request; on approve M5 issues XFR and writes back xfr_document_id	transfer_request.*.v1

6.1 ConfirmGrnHandler (the pivot) and the ingestion hook

```
@CommandHandler(permission = "shop.grn.confirm | whs.grn.confirm") // resolved by location type
@Transactional
GrnId handle(ConfirmGrn cmd, ScopeContext ctx) {
  Grn g = repo.draft(cmd.grnId()); require(g.receiverLocation in ctx.scope, "scope.mismatch");
  for (line : g.lines()) { // validation
    Sku sku = catalogue.getSku(line.skuId());
    require(line.receivedQty >= 0 && line.damagedQty <= line.receivedQty, "grn.line.quantities");
    if (sku.hasPrintedMrp && line.receivedQty > 0) require(line.printedMrp != null, "grn.line.mrp_required");
    if (sku.expiryTracked && line.receivedQty > 0) require(line.expiryDate != null, "grn.line.expiry_required");
    require(catalogue.conversionExists(sku, line.uom, today), "grn.line.uom");
  }
  // cost basis per line: trade price of the relationship for DN receipts (M3, tier by ordered qty), keyed
  unit_cost for local supply
  g.lines().forEach(l -> l.unitCost = g.isLocalSupply() ? l.unitCost :
    pricing.resolveTradePrice(g.relationship(), l.skuId(), l.uom, l.orderedQty(), today).price());
  // 1. issue: LOCATION series when receiverLocation is a SHOP, ENTITY series otherwise (kernel picks by
  document_type + location)
  documents.issue(g, ctx); // number, hash, state
  DRAFT->CONFIRMED, audit GRN_CONFIRMED
  // 2. batches (M2) - existing-or-create; synthetic when not batch-tracked
  for (line : g.lines()) if (line.receivedQty > 0)
    line.batchId = catalogue.registerBatch(new RegisterBatch(line.skuId, g.supplierId(), line.batchNo,
      line.manufactureDate, line.expiryDate, line.printedMrp, g.id()));
  // 3. the pivot event: M5 creates lots (qty received - damaged as GOOD, damaged as DAMAGED lot) and posts
  RECEIPT movements
  events.publish(GrnConfirmed.of(g)); // receiver, location, seller,
  drop, lines(batch, qty, damaged, unitCost), confirmedAt
  // 4. discrepancy
  if (DiscrepancyDetector.hasVariance(g)) { Discrepancy d = disputes.raise(g, ctx);
    events.publish(DiscrepancyRaised.of(d)); g.markDiscrepancy(); }
  else g.close();
  if (g.dropId() != null) drops.markReceived(g.dropId(), g.id()); // DN closes when all drops
  terminal
  return g.id();
}
// Offline path: the till already issued the GRN (number from the LOCATION series). The kernel bundle
  ingestion inserts the
// document, then calls GrnPostIngestHook.apply(grn, systemScope(device)) which runs steps 2-4 above
  (validation re-run; a
// violation is FLAGGED, never rejected - doc 32 S4; e.g. MRP missing -> line flagged, batch registered with
  null MRP, REVIEW).
```

6.2 AcceptOrderHandler and the allocation engine

```
handle(AcceptOrder cmd, ctx):
  o = repo.submitted(cmd.orderId); require(o.seller == ctx.entityId, "scope.mismatch"); rel =
    party.lookupRelationship(o.seller, o.buyer, today)
  avail = inventory.availability(sellerLocations(ctx.entityId), o.skuIds) // snapshot stored on the
```

```

run
run = AllocationEngine.run(rel.allocationRule, candidates = openOrdersForSeller(ctx.entityId, o.skuIds) + o,
    avail, overrides = cmd.overrides)
// FCFS: order candidates by submitted_at; for each line allocate min(remaining request, remaining avail);
// overrides replace a line's
// allocation and must carry a reason; result recorded in allocation_run.overrides; other rules (PRO_RATA,
// QUOTA) are strategies behind the same interface
for (l : o.lines) l.allocatedQty = run.allocationFor(l); l.tierPrice = pricing.resolveTradePrice(rel, l.sku,
    l.uom, l.requestedQty, today).price()
o.committedEta = cmd.eta; o.lockAt = cmd.eta.atStartOfDay().minusHours(rel.orderLockHoursBeforeEta);
o.status = ACCEPTED
audit.record(ORDER_ACCEPTED, o, null, o.summary(), ctx); events.publish(OrderAccepted.of(o, run.id));
events.publish(OrderAllocated.of(o))

```

6.3 Settlement, cheque bounce and exposure

```

RecordPaymentReceiptHandler.handle(cmd, ctx):
    rel = party.lookupRelationship(ctx.entityId, cmd.payer, today) (or customer account for M7's reuse)
    require(cmd.amount > 0); if (cmd.method == CHEQUE) require(cmd.cheque != null)
    sum = 0; for (s : cmd.settlements) { inv = billing.invoice(s.invoiceId); require(inv.seller == ctx.entityId
        && inv.buyer == cmd.payer);
        require(s.amount <= inv.openBalance(),
            "settlement.exceeds_open_balance"); sum += s.amount }
    require(sum <= cmd.amount, "settlement.exceeds_receipt")
    prc = documents.issue(PaymentReceipt.of(cmd, unapplied = cmd.amount - sum), ctx) // seller
    ENTITY series
    for (s : cmd.settlements) { links.link(prc, s.invoiceId, SETTLES, s.amount);
        billing.recacheSettled(s.invoiceId) } // status PARTIALLY/SETTLED
    if (cmd.method == CHEQUE) cheques.insert(prc, cmd.cheque)
    posting.publishReady(prc); events.publish(PaymentRecorded.of(prc)) // exposure
    consumer picks it up

RecordChequeOutcomeHandler(BOUNCED): ReversePaymentReceipt(prc, reason = "cheque.bounced") with MFA waived
    (system consequence):
    rev = documents.issue(PaymentReceipt.reversalOf(prc), ctx); links.link(rev, prc, REVERSES)
    for each SETTLES link of prc: links.link(rev, invoice, SETTLES, -amount); billing.recacheSettled(invoice)
    // invoices reopen
    audit ALERT CHEQUE_BOUNCED; events.publish(ChequeBounced, PaymentReversed)

@EventConsumer(types={"invoice.issued.v1","credit_note.issued.v1","credit_note.applied.v1","payment_receipt.recorded.v1",
    "payment_receipt.reversed.v1","order.accepted.v1","order.cancelled.v1","order.allocated.v1"},
    consumer="m4.exposure")
ExposureRecalculator.on(e):
    rel = e.relationshipId; x = exposure.rowFor(rel) FOR UPDATE
    x.openInvoices = sum(gross - settled - credited for INV ISSUED/PARTIALLY_SETTLED/DISPUTED of rel)
    // indexed query, not a scan
    x.acceptedNotInvoiced = sum(allocated - fulfilledInvoiced qty x tierPrice for ACCEPTED/LOCKED/PARTIALLY
        orders)
    x.unappliedCredits = sum(CN ISSUED unapplied); x.unappliedReceipts = sum(PRC unapplied_amount)
    x.amount = openInvoices + acceptedNotInvoiced - unappliedCredits - unappliedReceipts; x.asOf = now;
    x.freshness = min(lastSyncOf(buyer's locations))
    thresholds = config.list("trading.exposure_warn_thresholds"); crossed = highest t where amount >= limit *
        t/100 and t > x.warnedThreshold
    if (crossed) { x.warnedThreshold = crossed; events.publish(ExposureWarning(rel, amount, limit, crossed,
        freshness)) } // warn only (ADR-12)
    events.publish(ExposureUpdated(rel, x))

```

7. Queries, jobs, consumers, snapshot and the till bundle

Item	Implementation
Queries	As doc 24 §5.2; PARTY reads through the masking views; ListUnsettledInvoices uses the cached settled/credited columns; DeliveryPerformance compares committed_eta with the drop's arrived_at.
LockClockJob (5 min)	ACCEPTED orders with lock_at ≤ now → LOCKED; audit ORDER_LOCKED.
EscalationClockJob (hourly)	RAISED/UNDER_REVIEW discrepancies past window_ends_at + grace → ESCALATED; claims likewise; notify the arbiter queue.
BackorderReviewJob (nightly)	Orders with backorder older than trading.backorder_review_days → REVIEW audit event to the seller.
StatementJob (per frequency)	For each relationship due: StatementBuilder → A4Renderer → statement.generated.v1.
Consumers	m4.exposure (6.3); m4.claim-return: on claim.approved with return_required → nothing here (M5 consumes the same event); m4.transfer-xfr: on M5's xfr.issued.v1 write back xfr_document_id; m4.batch-correction: on batch.corrected.v1 nothing changes on issued documents (they keep mrp as recorded) – REVIEW only.

7.3 Snapshot contributor: expected lines for a shop

```
DropSnapshotContributor: tables(): ["expected_drop"]
rowsFor(location, ids): drops with ship_to_location = location and status PLANNED/DELIVERED (not yet
  RECEIVED):
  {dropId, dnNumber, sellerEntityId, sellerName, billToEntityId, dispatchedAt, eta, lines[{skuId, uom,
    dispatchedQty, batchId?, batchNo?, expiryDate?, printedMrp?}]}
change-log: on delivery_note.issued -> UPSERT per drop's shop (urgent when eta - today <= 1 day); on
  grn.confirmed / drop.undelivered -> TOMBSTONE
```

7.4 The GRN bundle the till sends (contract with M6)

```
event_type: grn.confirmed.v1 (bundle) source: device payload:
{ document: { documentId, docTypeCode: "GRN", seriesId (LOCATION series), docNumber, docNumberDisplay,
  ownerEntityId, counterpartyEntityId (seller or null),
    locationId, tillPositionId, deviceId, status: "CONFIRMED", issuedAt, issuedLocal, businessDate,
    operatorUserId, contentHash, origin: "OFFLINE", deviceSeq },
  grn: { dropId?, deliveryDocumentId?, supplierId?, receivedOn, countedBy, confirmedBy, confirmedAt },
  lines: [ { lineId, lineNo, skuId, uomCode, expectedQty?, receivedQty, damagedQty, batchNo?,
    manufactureDate?, expiryDate?, printedMrp?, unitCost? } ],
  stateHistory: [ {from: "DRAFT", to: "CONFIRMED", occurredAt, actor} ],
  attachments: [ {attachmentId, objectKey, contentHash} ] // photos of damage, uploaded via presign; may
    be PENDING
}
central: kernel inserts document/lines/state/attachments; GrnPostIngestHook runs validation (flag, never
  reject), batches, GrnConfirmed event, discrepancy.
```

8. Screens

```
Goods received (till, shop-management mode) - specification for M6
+-----+
+-----+
| ScopeBanner: Shop 07 | shop mode | shop-in-charge | OFFLINE | | Orders to accept (3) Allocation
| run: [Run FCFS v] |
| Delivery from Central Distributors DN D007-DN-004512 ETA today| | ORD M042-ORD-001234 Kandy South
| 12 lines Rs 2.6M |
| Line Item (thumb) Expected Received Damaged Batch/MRP| | Milk powder 400g req 100 avail
| 60 alloc [60] ! |
| 1 Milk powder 400g 10 CASE [10] [0] B2411A / 1080 | | override reason
| [ ] (required) |
| 2 Red rice 1kg 48 EA [46] [2] R77 / - | | ETA [2026-10-16] exposure after
| accept: 9.78M/10M ! |
| ... | | [Accept with allocations]
| [Reject...] |
| [Add unexpected line] [Photo] pending uploads: 1 |
+-----+
| [Confirm receipt] -> ownership transfers; number M042-S07-GRN-000217 |
+-----+
Rules for the till screen: expected lines come from the expected_drop snapshot row; received defaults to
  expected (one tap to accept);
MRP and expiry fields appear only when the SKU's flags require them and block Confirm when empty; damaged
  opens the camera;
Confirm is offline; the number is shown immediately (LOCATION series on this device when primary; else "will
  be numbered on sync" is NOT
```

acceptable - a non-primary till cannot confirm a GRN: it captures a DRAFT and hands to the primary till or the web).

Screen	Components	Message ids (m4.)	Notes
Requisition book	DocumentHeader/Lines, availability column, ApprovalBar, exposure FreshnessStamp	order.*	Amend disabled after lock with the reason
Order desk	list, allocation table with override reasons, StepUp none	order.*, allocation.*	As sketched
Delivery note	DocumentHeader, drops tabs, lines with batches from pick list, print	delivery.*	Driver assignment from M1 users of kind DRIVER
Drop list and POD (driver app)	stops list, signature pad, camera, offline queue	driver.*	Built by the till track from this spec
Goods received (web)	same as till spec on the web shell	grn.*	Warehouse receipts; local supply with supplier picker
Discrepancy / Claim	Timeline, evidence gallery, propose/accept forms, ReasonCapture	dispute.*	Two-step resolution visible as 'proposed by X, awaiting Y'
Tax invoice / credit note / debit note	DocumentHeader/Lines read-only, links panel, A4 print	invoice.*	PARTY view hides cost
Receipt book and matching	receipt form, invoice picker with open balances, cheque fields, StepUpDialog on reverse	payment.*	Unapplied amount shown
Statement / Exposure	rendered PDF viewer; components table with FreshnessStamp	statement.*, exposure.*	–
Arbitration queue	ExceptionQueue variant with both parties' statements and evidence; decision form with StepUp	arbitration.*	F only

9. Tests

Layer	Cases	Target
Unit	Every guard of Section 6; AllocationEngine FCFS with overrides; lock_at; InvoiceBuilder tax and tiers (ordered qty); open balance from links; exposure formula and thresholds; cheque bounce reversal arithmetic	All guards
Contract	Every operation; captureGrn example reproduces; PARTY invoice read has no cost columns; driver POD idempotent replay	Every operation
State machine	All seven machines of doc 24 §4 incl. two-step resolution and the clocks	All transitions
Property-based	Settled + credited never exceed invoice total under random link sequences; exposure equals recomputed-from-scratch after any event order; GRN received qty independent of DN qty; a reversed GRN leaves lots as before (via M5 stub)	1000 cases each
Bundle ingestion	GRN bundle from the till simulator: applied; MRP missing → flagged not rejected; duplicate resend → DUPLICATE; drop TOMBSTONE in the change log	Four cases
Scenario	Doc 13 scenarios 1 and 2 end to end with M2 real and M5 stubbed; flows 6.3–6.8 of doc 24	Eight flows
Performance	Allocation over 500 open orders under 10 s; exposure recompute under 50 ms; 800 statements under 15 min; invoice issue under 300 ms	Four targets
Web	Playwright: order → accept with override → DN → GRN (web) with discrepancy → propose/accept → credit note; record cheque → bounce → invoice reopens	Two flows

10. Tickets in build order

#	Ticket	Est. (days)	Done when
M4-01	Scaffold; migrations (extension tables, views, RLS incl. PARTY); seeds (permissions, posting map, config, notification rules)	2	Applies; RLS matrix rows
M4-02	Order: aggregate, create/submit/amend/cancel, queries, availability via M5 stub	2.5	Submit issues from buyer series
M4-03	Allocation engine and AcceptOrder/Reject/Allocate/LockOverride/Adjust; LockClockJob; BackorderReviewJob	2.5	Flow 6.1 to acceptance
M4-04	Delivery note with drops, issue, dispatch, POD (driver API), undelivered; DropSnapshotContributor	3	Drops in snapshot; POD offline replay
M4-05	GRN: capture, ConfirmGrnHandler with M2 real and M5 stub, DiscrepancyDetector, GrnPostIngestHook, reversal	4	Bundle ingestion tests; the pivot end to end
M4-06	Discrepancy: two-step resolution, consequences, EscalationClockJob, arbitration	2.5	Flow 6.2, 6.4
M4-07	Claims with attachments; consequences; escalation	2	Flow 6.3
M4-08	Invoice builder, issue, dispute; credit and debit notes with links and caches; posting mapper	3	Invoice from GRN at tiered price; postings_ready payload
M4-09	Payment receipts, settlement, cheque outcomes, reversal; exposure consumer and warnings	3	Property tests; bounce reopens
M4-10	Statements (template + job); transfer requests; delivery performance query	1.5	Statement PDF; XFR write-back via M5 stub
M4-11	Web screens (ten) and driver app spec hand-off; Playwright flows	5	Two flows green
M4-12	Contract freeze: m4trading.yaml v1; api/query review with M5, M6, M7, M9; doc 33 slice; till bundle contract signed by the till track	1	Dependants acknowledge

Total about 32 developer-days. Freeze the GRN bundle contract (7.4) and the expected-drop snapshot row (7.3) at M4-05, before the rest of the module, because M6 builds against them.

11. Open points: what to assume, whom to ask

Point	Assume	Decider	If different
Credit note from an accepted discrepancy proposal without MFA (DR-1 here)	MFA waived because both parties agreed in the two-step flow	Federation admins	Add step-up to AcceptResolution for the seller

Point	Assume	Decider	If different
Invoice timing, tier basis, reversal and claim windows, escalation grace (doc 24 DR-1 to DR-6)	Defaults of doc 24 §7	Federation admins	Config values; no code change except invoice-on-dispatch
Direct drop without snapshot data (DR-7)	Capture as local supply keyed against the DN number; reconcile at ingestion	M4, M6	–
Non-primary till confirming a GRN	Not allowed: DRAFT only; primary till or web confirms	Till track	Location series holder becomes any till (doc 32 counter transfer semantics change)
Numbering (24B)	No FY reset; GRN non-fiscal; discrepancy/claim in issuer ENTITY series	Tax advisor	Series definitions in the kernel seed; handlers unchanged
M5 queries (availability, lotsConsumed, pickBatches)	Stubs with fixtures until doc 25	M5 developer	Swap
Posting map account roles	As seeded; chart mapping later	Accountant review (doc 20 M9 risk)	Seed edits only

End of document. 24A v0.1; m4trading/README.md and its tests become the living version once merged.